

A* Search for Road Routing and Partially Observed Robot Navigation: Design, Implementation, and Experimental Evaluation

Nguyen Hoang Thang

Faculty of Computer Science and Engineering
University of Technology
Vietnam National University, Ho Chi Minh City

Abstract. Heuristic search solves known shortest-path problems while remaining one component of navigation in unknown environments. This report presents reusable C++17 A* integrated with directed road routing and partially observed robot navigation. The robot accumulates limited-range observations, selects exploration targets, repeatedly plans on a verified visibility graph, and returns to observed ancestors. Recorded reversible entry motion bounds executed retreat under explicit assumptions. Evaluation uses a frozen benchmark. On 180 directed road queries, A* and Dijkstra return equal costs; A* reduces expansions by a median paired 76.0% and has a median search-time ratio of 0.338. A separate comparison retains 21 nontrivial local requests from 351 source decisions. The original authors' approximate-path procedure and our A* adapter both produce obstacle-interior-valid point paths in every retained request, with zero median paired length difference. Different graphs, boundary conventions, and timing stages limit stronger comparisons. Supporting studies examine exploration, sensing, frontier caching, and certified returns. The implementation provides a reproducible case study of graph-optimal planning and conditional recovery under partial observability.

Keywords: A* search · Road routing · Partial observability · Visibility graph · Blind-alley recovery · Reproducible evaluation

1 Introduction

The information available to a planner determines which paths it can evaluate and optimize. Known road topology and edge costs define a fixed optimization problem. A robot with limited vision must also choose where to obtain observations before planning through unknown passages. A locally optimal path may lead into a dead end, requiring return to an observed branching position. Graph search and exploration therefore address distinct, related problems.

This project investigates one C++ implementation of A* in both settings. Road Navigation minimizes distance on a static directed graph. Blind-Alley Robot Navigation plans continuous piecewise-linear motion between exploration

targets and returns to observed ancestors using accumulated polygonal observations. Python supplies sensing and exploration around the shared engine. An educational Robot Lab and four-direction blind-alley fixtures remain validated baselines.

Phan et al. [8] investigate navigation through blind-alley regions and analyze return-path properties under stated geometric assumptions. Their method represents explored space through bundles of line segments and refines paths geometrically. Here, recorded reversible motion supplies a feasible return while A* seeks a shorter verified graph path. This alternative requires explicit sensing and execution assumptions. Comparison must also distinguish computed approximate paths from recorded simulator coordinate transitions.

Accordingly, we examine three questions: how adapters preserve search assumptions; what effort and time geographic guidance saves relative to Dijkstra on the same road graph; and how local robot paths differ at identical observed states and endpoints. The last comparison accounts for representation, validity conventions, and computation stages. Separate online studies examine exploration ordering and executed recovery.

The contribution is a reproducible implementation and controlled evaluation of shared graph search, together with a conditional executed-return certificate. Its scope is established A* theory and observed-graph planning; global unknown-world optimality is not established. Measurements use frozen revision 04c4846.

2 Theoretical Background and Related Work

2.1 Shortest-Path Search and Heuristic Assumptions

Once a graph is available, both applications require minimum-cost paths. On directed $G = (V, E, w)$ with nonnegative weights, Dijkstra orders pending states by tentative start cost [2]. A* adds estimated remaining cost to guide search toward the goal [4]:

$$f(n) = g(n) + h(n). \quad (1)$$

Here $g(n)$ is the best start-to- n cost found so far and $h(n)$ estimates cost to goal t . OPEN contains pending states; improvements update costs and predecessors. The goal is accepted on valid minimum-priority extraction, not discovery. Setting $h = 0$ gives Dijkstra in the same implementation.

This stopping rule requires suitable estimates. Admissibility means $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is minimum remaining graph cost. Consistency additionally requires

$$h(u) \leq w(u, v) + h(v) \quad \text{for every } (u, v) \in E, \quad h(t) = 0. \quad (2)$$

Consistency makes the first valid expansion's cost optimal. Admissible but inconsistent estimates can require reopening improved states. These distinctions and tie resolution underlie Hart et al. [4] and the implementation in Section 5. Fewer expansions need not mean lower wall time: heuristic evaluation and representation also incur work.

2.2 Geometric Representation and Replanning

Optimality depends on admitted paths. Grids restrict headings and may miss narrow passages; visibility-based planning uses obstacle-free geometric connections [7]. Finite footprints require configuration-space treatment. A complete polygonal visibility graph and a sparse scan-center graph admit different paths, so optimality in the latter need not imply a continuous shortest path.

Theta* uses grid search with off-grid connections [1]. Experiments on random grids and game maps compare length, time, expansions, and heading changes, finding shorter paths than grid smoothing without a full visibility graph. Basic Theta* nevertheless does not guarantee a continuous shortest path. Our ordinary A* on verified continuous links likewise requires separate assessment of representation and search.

When observations change, D* Lite reuses search information through locally consistent value updates [6]. Its unknown-terrain navigation and mapping experiments reduce operations relative to repeated search. Those grids initially allow unknown traversal optimistically; our graph admits only verified free links. Repeated A* preserves implementation transparency. Incremental replanning remains an alternative whose project performance is unmeasured.

2.3 Observation, Exploration, and Escape

Constructing links first requires representing sensor knowledge. Elfes accumulates uncertain evidence in probabilistic occupancy grids [3]; sonar and stereo demonstrations show the value of additional viewpoints. Our deterministic labels and polygon unions accumulate observations and preserve unknown space, but omit probabilistic uncertainty and localization drift.

Free/unknown boundaries then suggest exploration targets. Yamauchi visits accessible frontiers [9]; office-robot experiments address sensor artifacts, while idealized coverage remains conditional on sensing and reachability. Our polygon candidates similarly seek observations, using ancestry for recovery and C++ A* for routing. Estimated utility does not prove information gain, and one unreachable candidate does not exhaust a branch.

TangentBug combines range discontinuities and a local tangent graph with target motion and boundary following [5]. Its convergence analysis assumes static finite-perimeter geometry and ideal point motion; selected simulations compare it with VisBug. Our simpler observed-parent policy does not inherit that guarantee. Together these studies explain why local path optimality requires a separate exploration and escape mechanism.

2.4 Bundle-Based Planning and the Assigned BAR Problem

Phan et al. [8] accumulate sights, rank active open points, construct a skeleton graph, and shorten polylines through ordered bundles of line segments. DAP adjusts vertices on critical segments within represented explored space; Section

3 relates approximation quality to geometric and numerical assumptions. Section 4.5 constructs bundles and Algorithm 2 integrates observations, ranking, refinement, and movement. The main contribution is geometric representation and refinement, not ordinary graph search.

Algorithm 2 is explicitly not generally convergent. Section 5 studies a special algorithm with obstacle-diameter and near-goal connectivity assumptions, which do not prove completion of our maze exploration. Section 6 compares local paths with a grid A* baseline and RRT*, and further evaluates RRTX. Dijkstra is discussed as a foundational search method rather than evaluated as an independent experimental baseline. The source’s `Graph.BFS_skeleton_path` performs Dijkstra-style cumulative-cost priority-queue search, despite its BFS-oriented name; this skeleton search precedes bundle-based refinement.

These differences determine the scope of our comparison: observations and endpoints are held fixed, while graph representations, boundary rules, and motion semantics are disclosed. The application addresses exploration with limited sensing and conditionally certified return to observed ancestors. It reproduces neither bundle optimization nor general polygonal BAR detection. The supplementary literature matrix records the relevance of all eight supplied PDFs, representing seven unique studies.

3 Problem Formulation

3.1 Known Directed Road Routing

Let $G = (V, E, w)$ be a finite directed road graph with finite nonnegative weights $w : E \rightarrow \mathbb{R}_{\geq 0}$. For origin $s \in V$ and destination $t \in V$, the routing objective is

$$P^* \in \operatorname{argmin}_{P: s \rightsquigarrow t} C(P), \quad C(P) = \sum_{(u,v) \in P} w(u,v). \quad (3)$$

Here $C(P)$ sums exported road lengths in meters. Directed edges enforce one-way topology; traffic, time-dependent costs, and turn restrictions absent from pre-processing are excluded. If no path exists, search reports failure. Node endpoints isolate search from coordinate snapping in the principal experiment. Interactive requests instead use projected road positions and temporary partial edges (Section 4).

3.2 Partially Observed Robot Navigation

Let $W \subset \mathbb{R}^2$ be a bounded workspace and $O \subset W$ the static polygonal obstacles. At controller decision k , define robot position x_k , goal q , accumulated observed free space F_k , and visible obstacle fragments B_k . The simulator uses O for sensing and motion validation; the policy receives observed geometry and traversal history. Its unknown region is $U_k = W \setminus (F_k \cup B_k)$. Range $R > 0$ and occlusion limit observations: membership in the sensing disk alone establishes neither visibility nor traversability.

The controller selects target z_k from observed information and history, then constructs $G_k = (V_k, E_k)$ from scan centers and request endpoints. Only wholly verified free segments become links. Edge weights $\|u - v\|_2$ and heuristic $\|u - z_k\|_2$ let A* minimize graph-path length. Execution checks collision and observed-free coverage. For consecutive executed positions $y_0, \dots, y_M$, the complete journey has length

$$L_{\text{online}} = \sum_{j=0}^{M-1} \|y_{j+1} - y_j\|_2. \quad (4)$$

This quantity includes both exploration and return motion. The index j counts executed segments, whereas k counts controller decisions; neither counts rendered frames. Consequently, L_{online} need not equal one local plan’s cost or the shortest distance available with full-map knowledge.

The continuous simulator uses an idealized point center and reversible motion along straight segments. Heading changes freely at vertices; acceleration, nonholonomic constraints, and disk clearance are absent. The frozen local comparison permits boundary tangency in its primary convention and separately reports strict no-contact validity. Neither convention validates the source’s nominal radius-0.5 parameter.

3.3 Recovery and Evaluation Objectives

A branch records an observed anchor and its subsequent executed trajectory. Exhaustion of usable local candidates triggers return to an observed ancestor with remaining alternatives. Failure to reach one target instead defers it for reconsideration after knowledge changes; it does not certify branch exhaustion. Exploration resumes after return. Section 5 derives the conditional executed-length bound.

Road evaluation measures minimum graph cost, valid expansions, and search-call time. Local robot evaluation measures planned Euclidean length, geometric turns, endpoint agreement, observed-space coverage, obstacle validity, and computation components. Supporting online studies measure actual distance, goal status, frontier processing, and executed returns. A return to an observed ancestor is *certified* only when the recorded entry trajectory is reversible and the executed return satisfies the prescribed endpoint and length constraints. Reaching a goal, ending with no eligible frontier, and exhausting a budget are different outcomes; none automatically proves a complete map or global unknown-world optimality.

4 System Architecture and Design

4.1 Shared Engine and Application Boundaries

Figure 1 separates shared search from application knowledge. React/TypeScript requests pass through FastAPI and pybind11 to C++17. Road loading and

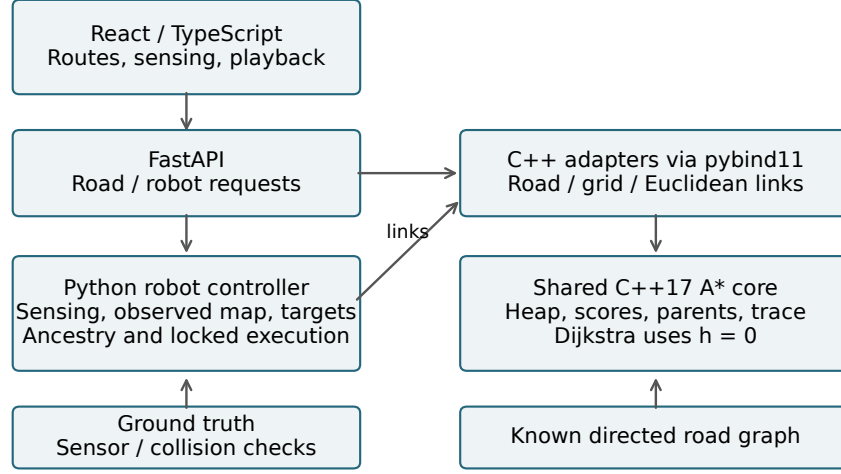


Fig. 1. Implemented system boundaries. Ground truth enters sensing and motion validation; the continuous planner receives observations and verified links. Traces support frontend playback.

adapters are C++; Python handles continuous sensing, accumulated geometry, targets, ancestry, and execution. Python constructs the continuous graph; the binding validates points and links and assigns Euclidean costs. Search therefore needs no hidden obstacle geometry.

Reuse requires a common problem contract. `include/astar/search.hpp` accepts neighbors, heuristic, goal predicate, and state hashing/equality, maintaining costs, predecessors, counters, and traces without coordinate or equal-cost assumptions. Disabling the heuristic gives Dijkstra in the same core, isolating guidance in the road comparison.

4.2 Road Representation and Coordinate Requests

The OSM-derived graph contains 2,986 vertices and 7,152 directed edges. Adjacency determines legal travel; polyline metadata supplies geometry (Fig. 2). Snapping a click between intersections to a vertex would shift the endpoint. Queries instead scan segments and project using latitude-scaled coordinates. Cumulative Haversine lengths determine edge fraction $r \in [0, 1]$; projection remains a local approximation.

For directed edge $u \rightarrow v$ of weight w , a snapped start connects forward with $(1 - r)w$, and a snapped goal connects from u with rw . Reverse travel requires a corresponding reversed polyline and uses its own weight and fraction. Two ordered positions on one directed edge obtain a direct connection of $(r_g - r_s)w$ when $r_g \geq r_s$. Endpoints reuse real nodes. Virtual states and partial edges belong

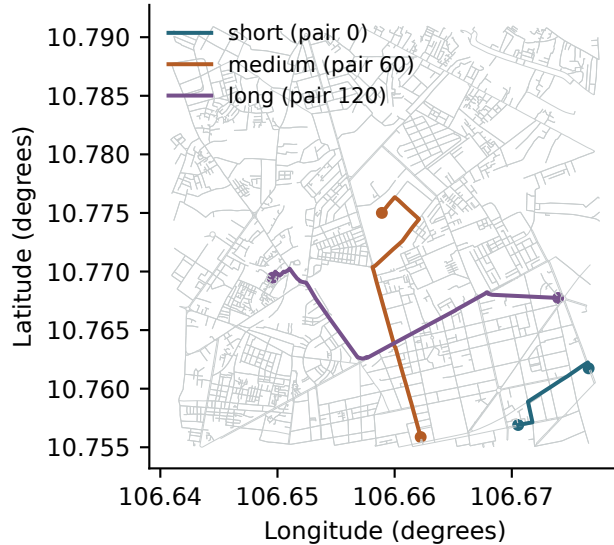


Fig. 2. Representative frozen short, medium, and long road queries. Background is the exported graph, not additional routing information; each highlighted route uses the unchanged C++ adapter. Axes use exported geographic coordinates.

only to the request adapter; the base graph is unchanged. Partial route geometry is sliced along the original polyline. The heuristic scale is also reduced if a partial edge requires it. Thus floating-point or geometric approximations do not justify blindly reusing an invalid lower bound.

4.3 Sensing, Exploration, and Observed Ancestry

Robot Lab supports four or eight directions (diagonal cost 1.5, without corner cutting), and grid BAR uses four unit-cost directions. Continuous polygon, seeded-maze, and indoor modes admit arbitrary straight-link headings, but require conservative observation geometry.

The sensor uses 72 uniform directions and rays near visible vertices. First-hit range, blocker margins, and checked free triangles approximate visibility before union into F_k . Occlusion distinguishes this region from the sensing disk. The policy receives observations without hidden polygons or topology; grid evaluation gates are consulted only after navigation. Ground-truth annotations are used only for evaluation and do not influence target selection or recovery decisions.

Traversal history enables recovery. Each exploration node records position, history index, branch ID, parent ID, and remaining candidates; a stack tracks ancestry. Rounded target identities prevent immediate repetition, while map or observation changes permit reconsidering deferred targets. Radar ranking estimates frontier opportunity from known boundaries and discounts direct Eu-

clidean travel. This proxy is neither a C++ A* path cost nor guaranteed information gain. Low gain does not exhaust a branch; a conservative fallback retains uncertain occluded candidates.

Recovery transfers deferred child alternatives, preserves siblings, and locks execution to the anchor. Graph-blocked targets remain distinct from exhausted branches. Playback displays sensing, plans, movement, branches, and returns, with graph debugging hidden by default. Search traces expose g, h, f and predecessors; continuous frames describe higher-level events. Display interpolation introduces no executed motion.

5 A* Algorithm Implementation

5.1 Queue, Records, and Improved Paths

Preserving improvements to pending or expanded states requires separating records from queue entries. An `unordered_map` stores best g , predecessor, and last-expanded g per state; a `priority_queue` stores state, inserted g, h, f , and insertion order. Lexicographic (f, h, order) selects lower f , lower h , then earlier insertion, deterministically when neighbor enumeration is deterministic.

Algorithm 1 pushes a new entry for each improvement rather than decreasing a key. Entries with stored g above the current best record, or no better than the last-expanded score, are skipped. The last-expanded g -score records expansion history without permanently closing a state, thereby allowing reopening when a strictly better path is found.

Algorithm 1 Shared C++ search (trace and counters omitted)

Require: Problem neighbors, heuristic, goal predicate; start s , goal t

- 1: Initialize records with $g = \infty$, no parent or expanded value
- 2: $g[s] \leftarrow 0$; push $(s, 0, h(s), h(s), \text{order}++)$
- 3: **while** OPEN is not empty **do**
- 4: Pop minimum entry e with state u
- 5: **if** $e.g > g[u]$ or $(u \text{ expanded and } e.g \geq g_{\text{expanded}}[u])$ **then**
- 6: **continue**
- 7: **end if**
- 8: $g_{\text{expanded}}[u] \leftarrow e.g$
- 9: **if** problem goal predicate holds for u, t **then**
- 10: **return** predecessor chain reversed, cost $e.g$
- 11: **end if**
- 12: **for** outgoing (u, v, w) **do**
- 13: Validate finite $w \geq 0$; $a \leftarrow e.g + w$
- 14: **if** $a < g[v]$ **then**
- 15: $g[v] \leftarrow a$; $\text{parent}[v] \leftarrow u$
- 16: Validate $h(v)$ and finite sum; push $(v, a, h(v), a + h(v), \text{order}++)$
- 17: **end if**
- 18: **end for**
- 19: **end while**
- 20: **return** not found, empty path, infinite internal cost

The subsequent goal test prevents stale entries from terminating search. Reconstruction follows predecessors to the parentless start and reverses them. Start equals goal yields a zero-cost singleton; failure yields an empty path and internal infinity, serialized as nullable cost. The engine rejects nonfinite or negative weights/estimates and overflow. Relaxation uses strict improvement, with tolerances only in external validation. Adapters remain responsible for admissibility.

For a finite graph, consistent heuristic, binary heap, and expected constant hashing time, lazy entries give a conservative $O((|V| + |E|) \log(|V| + |E|))$ time and $O(|V| + |E|)$ auxiliary-space bound. Geometry and traces add costs. Inconsistent estimates may reopen states, invalidating a single-expansion bound.

5.2 Application Heuristics and Trace Semantics

Heuristics follow the movement model. For roads, let d_H be Haversine distance in meters. The loader chooses $0 \leq \alpha \leq 1$ with $\alpha d_H(u, v) \leq w(u, v)$ on every edge and sets $h(u) = \alpha d_H(u, t)$. The spherical triangle inequality gives $h(u) \leq \alpha d_H(u, v) + h(v) \leq w(u, v) + h(v)$, establishing Eq. 2. The frozen graph retains $\alpha = 1$; this bound requires no reverse edges.

For absolute grid differences dx, dy , four-direction unit movement uses Manhattan distance; eight-direction Robot Lab uses $\max(dx, dy) + 0.5 \min(dx, dy)$ for its diagonal cost of 1.5. Euclidean continuous-link weights make Euclidean goal distance consistent by the ordinary triangle inequality. Each guarantee thus concerns the adapter's admitted paths.

Expanded nodes count valid pops, including goal and re-expansions; generated nodes count all pushes. Unique counts are separate. Examined and relaxed edges count scanned neighbors and improvements. Trace events expose discovery, updates, expansion, closure, and goal scores. Stale entries are skipped; timing disables traces.

5.3 Executed Return Certificate

Recovery requires an executed-motion bound. Let $T = (a = p_0, \dots, p_m = b)$ be recorded entry from anchor a to current b , with $L_T = \sum_{i=0}^{m-1} \|p_{i+1} - p_i\|_2$. Under static obstacles, sound observed-free checks, and reversible point motion, its reverse is feasible with equal length. If recorded waypoints and links appear in the planning graph, graph-optimal A* gives $L_{A^*} \leq L_T$. Acceptance additionally checks endpoints, segment coverage, and length. Planning failure or an excessive length invokes the separately verified recorded reverse.

To preserve the bound during execution, the controller completes the accepted return without switching to exploration. For measured length L_R and final position x_{end} , it requires

$$L_R \leq L_T + \varepsilon, \quad \|x_{\text{end}} - a\|_2 \leq \varepsilon, \quad (5)$$

Here ε represents numerical length and endpoint tolerances in world units. Sensing may add observations without changing the destination. Certification requires verified reversible entry and locked execution; it does not establish semantic BAR boundaries, disk clearance, universal detection, or completeness. Reversed-entry fallback can satisfy the bound without an A*-computed shortcut, so outcomes are recorded separately.

6 Experimental Methodology

6.1 Frozen Inputs and Controlled Road Comparison

The experiments distinguish heuristic guidance on a fixed graph, local geometry at shared observed states, and online exploration/recovery. Only the first isolates a search algorithm change; the others require representation and execution qualifications. Inputs and results are frozen at `04c4846`, with manifest hashes and an unmodified authors' checkout at `8bbbfb8`.

The recorded host is Windows 11, Intel Core 7 240H (16 logical processors), with approximately 15.7 GiB RAM. CMake builds C++17 Release using MSYS2 UCRT64 g++; the binding runs in Python 3.12.10, original ASP in Python 3.14.2. Different environments and uncontrolled frequency/load make timings descriptive, not algorithm-only comparisons.

To examine route-length dependence, selection sampled 6,000 directed candidates with seed 20261009; 5,965 were reachable. Dijkstra ranks define strata at approximately 2,086.98 and 3,273.44 m, retaining 60 queries each for 180 frozen benchmark queries.

Only the heuristic flag changes between methods on the same graph and engine. One untimed call precedes seven alternating-order batches of 20, 10, and 5 calls in the respective strata. Time is the median batch mean per call. Loading, snapping, HTTP, rendering, and traces are excluded; common timed status/cost checks remain.

6.2 Source Audit and Shared-State Robot Requests

Compatible motion semantics are necessary for end-to-end comparison. The source logs an ASP but updates coordinates to its selected open point without ASP interpolation; Algorithm 2 specifies DAP motion. Straight-chord interpretations produce four interior crossings in the dead-end reproduction and one in the bugtrap, while planned ASPs avoid interiors. Coordinate transitions alone do not establish physical trajectories. The 48-configuration matrix therefore remains blocked and unexecuted: its entries are not failures, and chord distances cannot be compared with our executed travel.

A local comparison avoids inferring motion: requests freeze position, target, sights, skeleton, ASP, and provenance. The original routine is replayed unchanged. Our A* uses identical endpoints and sights, with analytic whole-link certification in a scan-center graph. This adapter preserves circular boundaries rather than the application’s polygon approximation. Full polygons enter only post-plan validation, never planner decisions.

Eligibility precedes local outcomes and requires more than two skeleton vertices, selecting nontrivial bundle refinement. Cohorts contain 43, 75, and 233 decisions (Table 1). Prospective selection preceded ten episodes at radii 10/20/30, capped at 60 iterations; capped outcomes remain. Correlated decisions and overlapping prior held-out bugtrap geometry limit generalization.

Table 1. Frozen robot decision accounting. Direct exclusions do not invoke bundle refinement; they are retained in the raw audit.

Cohort	Captured	Eligible	Excluded
Initial	43	6	37
Prior held-out	75	1	74
Prospective extension	233	14	219
Combined	351	21	330

Direct exclusions comprise 176 source-visible and 154 strict-range-boundary cases. Boundary convention explains endpoint predicate differences, with no missing-input or eligibility-checker mismatch. All 21 eligible requests remain, regardless of planner outcome.

6.3 Validity, Metrics, and Timing Boundaries

Primary robot validity requires matching endpoints within 10^{-7} , full observed-sight segment coverage, and no polygon-interior entry; boundary contact is permitted. Strict point validity rejects boundary contacts as well. Radius-0.5 clearance is a separate offline diagnostic, not the planned collision model. Event-based segment certification uses tolerance 10^{-9} ; sampled predicate comparisons supplement it rather than prove universal sensor equivalence. Across all cohorts, 175,500 membership samples and 111,672 graph-link samples have no unexplained mismatch or uncovered sample. All 865 recorded ASP segments are sight-certified; 1,128 accepted A* graph links avoid polygon interiors and boundaries offline.

Length is a sum of Euclidean polyline segments. An all-turn count uses wrapped heading changes exceeding 10^{-6} radians; an interpretable significant-turn count uses changes greater than 15° . Neither is a motor command count. ASP critical-line counts and A* expansions are not interchangeable operations. Robot component timing uses three warm-ups and 31 measured calls per state, then medians across state medians. ASP core excludes source skeleton generation; A* graph construction includes observed-link certification and is separated from its binding/search call. Medians of components need not sum to a median pipeline time.

Separate online studies retain 20 legacy/radar pairs, 20 full/cached frontier pairs, 12 sensor episodes, and an indoor parent trace. They measure our executed behavior, not local planning against the authors. Statistics are paired by configuration before aggregation, without significance or population-superiority claims. Report scripts leave frozen results unchanged.

7 Results and Discussion

7.1 Road Cost Agreement and Search Effort

All 180 matched road queries return equal costs under A*, Dijkstra, and the frozen offline reference within 10^{-6} m. The graph validator reports zero heuristic edge violations and scale 1.0. Separately retained edge cases cover start equals goal, unreachable targets, directionality, and edge-midpoint snapping. These checks support correctness on the tested graph; agreement alone is not a proof for every possible adapter.

Table 2. Frozen road results; 60 pairs per stratum. Expansion and time columns are marginal medians; reduction and ratio are medians of paired statistics.

Stratum	Expanded A*/D	Reduction	Time A*/D (μ s)	Ratio
Short	101 / 505	78.6%	44.95 / 128.09	0.324
Medium	301 / 1,592	77.4%	282.95 / 1,117.24	0.287
Long	714.5 / 2,588.5	68.6%	771.46 / 1,883.19	0.449

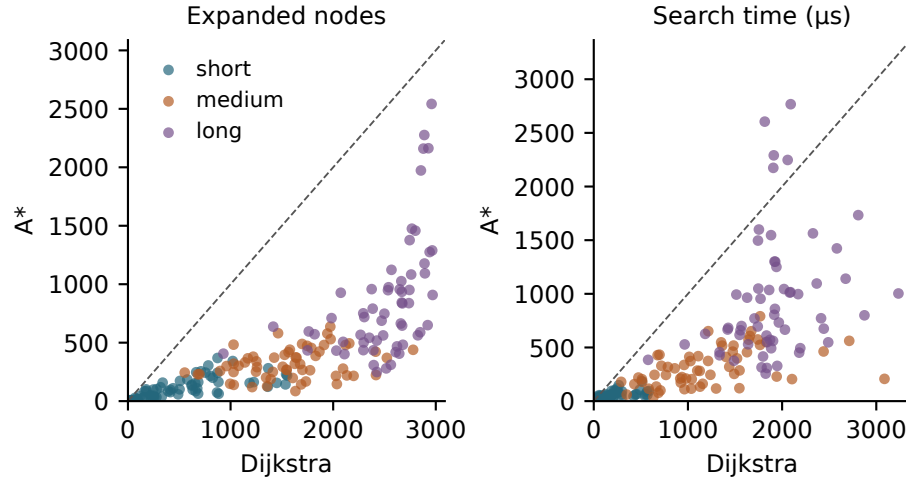


Fig. 3. Matched road search effort and search-call timing. Points below the diagonal favor A*. Each stratum has 60 queries; all 180 costs agree. Timing excludes graph loading, snapping, API, and rendering.

Table 2 and Fig. 3 show fewer expansions in every A* run, with median paired reduction 76.0%. Geographic guidance directs search toward the destination while preserving the cost objective. A* is faster in 173 cases, Dijkstra in seven; the median paired time ratio is 0.338, range 0.061–1.435, with pair 167 the largest regression. These data support substantial savings on this network, not universal timing dominance. Paired ratios differ from ratios of marginal medians. The shared graph controls representation more tightly than the robot study.

7.2 Robot Local Geometry and Path Length

Both local planners find paths for all 21 eligible requests, with correct endpoints and primary interior-only validity in all 21. These are local planning successes, not 21 successful autonomous missions. Of the ten prospective source episodes, nine reach the goal and one reaches its iteration cap; its eligible states remain. Figure 4 shows an observed-space request and the two returned paths. No hidden polygons are supplied to either local planner.

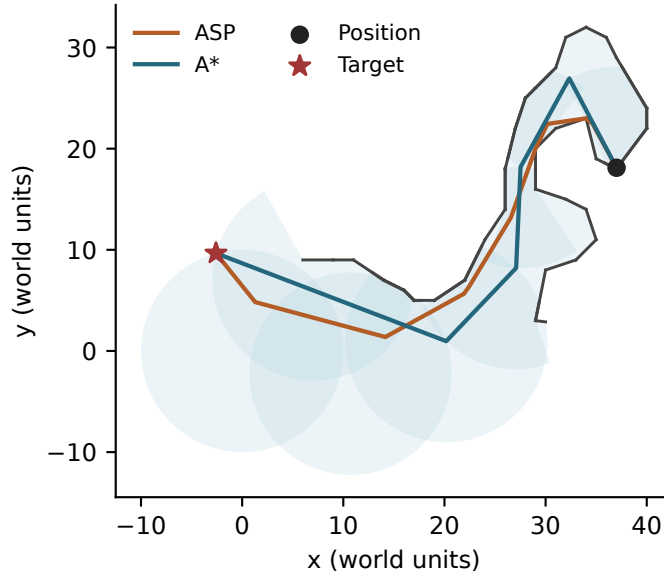


Fig. 4. Representative frozen dead-end request. Visible blocker fragments and approximate display sight footprints show recorded knowledge; ASP and scan-center A* use the same position and target. Sight footprints are display approximations; benchmark link admission uses analytic whole-segment certification. Hidden obstacles are omitted.

Table 3. Local robot outcomes by cohort. “Shorter” counts follow the primary point convention. Two prospective cases have equal lengths. Delta is the median paired A* minus ASP length in world units.

Cohort	Pairs	ASP shorter	A* shorter	Delta
Initial	6	5	1	+7.403
Prior held-out	1	1	0	+1.654
Prospective	14	3	9	-2.492
Combined	21	9	10	0.000

Define paired difference as A* length minus ASP length; positive values favor ASP. ASP is shorter in nine requests, A* in ten, and two are equal. The median is zero, mean +2.981, and range $[-10.707, +44.266]$ world units. Different representations admit different paths, and neither planner consistently dominates. Cohort composition matters: five of six initial cases favor ASP, nine of fourteen prospective cases favor A*. Figure 5 retains all pairs; one held-out request cannot establish generalization.

Boundary conventions explain part of this variation. ASP passes strict validity in 15 of 21 requests and A* in all 21. All six boundary-touching ASP paths

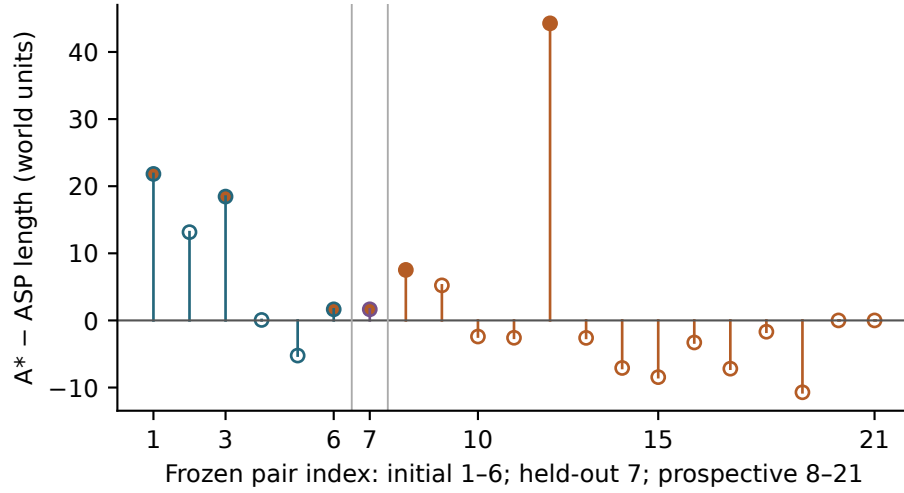


Fig. 5. All 21 eligible local requests, grouped by frozen cohort. Positive A* minus ASP differences favor ASP. Filled orange markers indicate ASP boundary contact under the strict convention; these paths remain primary interior-valid.

are shorter under the permissive convention. A dead-end configuration with the largest difference (P02-`_map_deadend-01`) yields 15.734 units for ASP versus 60.000 for A*, with ASP boundary contact. Among 15 jointly strict-valid pairs, ASP is shorter in three, A* in ten, and two tie; median difference is -2.584 . This secondary criterion cannot replace the primary result. Geometric refinement and strict sparse-center links admit different routes, requiring joint interpretation of length and validity.

Radius-0.5 clearance passes only nine ASP and eleven A* paths, precluding disk-safety claims. Significant-turn medians are two and one respectively; paired all-turn delta median is -2 . Tiny ASP bends can inflate all-turn counts without corresponding physical steering. ASP vertices can lie inside bundles absent from the sparse A* graph, so mixed lengths do not contradict graph-optimal search.

7.3 Robot Computation Components

Figure 7 separates the measured components. Across state medians, ASP core time is 62.655 ms, critical-line construction 0.319 ms, and optimization 62.400 ms. A* link-build time is 42.102 ms, binding/search 0.0263 ms, and the median per-state build-plus-call sum 42.142 ms. The small C++ call is a search on already constructed sparse graphs, while Python geometric certification can dominate its surrounding pipeline.

Construction and certification dominate the small C++ call in these requests. Dividing ASP core time by that call would compare different languages and preparation; even build-plus-call and ASP core omit different upstream work.

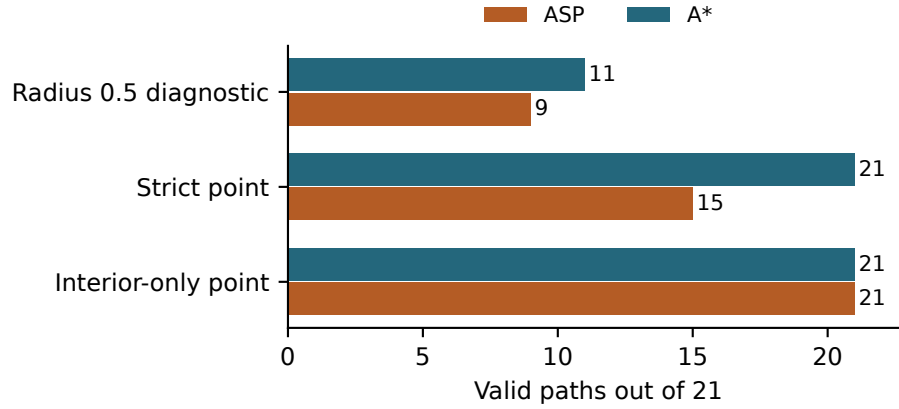


Fig. 6. Validity outcomes for the same 21 requests. Primary point validity allows boundary tangency; strict validity forbids it. Disk clearance is only a post-plan sensitivity check, not a guarantee of executed finite-robot motion.

Thus the evidence locates computational cost without isolating intrinsic algorithm speedup. Stronger timing conclusions require matched pipelines.

7.4 Supporting Online Behavior and Recovery

Online navigation adds target ordering. Both legacy and radar reach the goal in 20 paired episodes; radar travels less in 18 and more in two, with median difference -95.419 world units. Its largest regression is $+70.772$ in Complex Irregular Labyrinth at radius 7. Estimated information changes branch ordering, with useful but non-dominating outcomes; low scores cannot justify exhaustion. All twelve sensor episodes reach their goals, but radius-5 and radius-7 distance medians are 92.909 and 113.278 units. Greater range need not shorten travel in this sample because it also changes decisions.

Caching addresses computation without changing exploration ordering. Full and cached construction preserve playback, recovery, status, and distance in all 20 pairs; median accumulated frontier times are 639.166 and 46.723 ms, respectively. These are episode component totals, not end-to-end timings. Avoiding repeated construction explains the reduction, but equivalence requires invalidation when new free geometry or walls change the mask. Evidence supports the tested indoor and maze configurations, not arbitrary geometric updates.

The four certified indoor returns in Fig. 8 are explicitly nonfallback: they demonstrate participation of C++ A* in recovery rather than automatic reversal of entry. Certification checks executed length to the observed ancestor under the assumptions of Eq. 5. Successful uncertified exits, unreachable-goal fixtures, and budget termination remain separate outcomes. In particular, a continuous no-frontier status establishes exhaustion of the controller’s candidates, not unreachability in the unknown world.

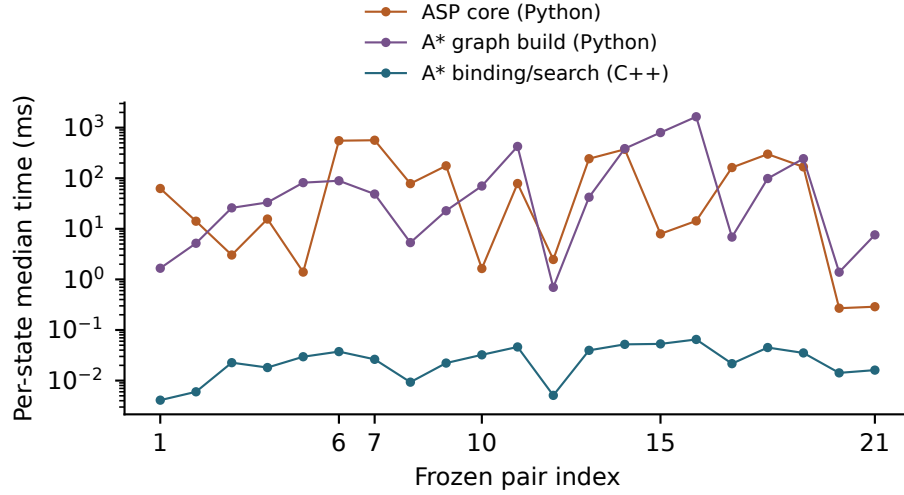


Fig. 7. Per-state median local computation times (three warm-ups, 31 calls). A logarithmic axis exposes component scale. ASP core excludes skeleton generation; A* build includes observed-link certification. Components and languages differ, so isolated search ratios are not algorithm-only speedups.

8 Limitations and Threats to Validity

Model validity. Static obstacles, ideal localization, reversible point motion, and reliable observations exclude probabilistic sensing, pose drift, dynamics, and footprint-aware configuration space. Sampled sensing may miss narrow occluded regions; accumulated free space differs from the sensing disk. Finite frontier candidates and budgets preclude inferring completeness in arbitrary polygonal worlds from fixture success.

Scope of guarantees. Missing geometric vertices can make graph-optimal paths longer than continuous optima. The return bound requires verified reversible entry, appropriate endpoints, and locked execution; it proves neither universal BAR identification nor optimal online travel. The paper’s approximation and restricted convergence arguments do not transfer to this controller. Our BAR-inspired application retains the stated restrictions rather than reproducing all published mechanisms.

Comparison validity. Shared observations do not imply shared graphs. Permissive and strict boundaries change interpretation; point validity does not establish disk safety. Source transitions and ASPs do not expose compatible continuous execution, keeping end-to-end comparison gated. Our adapter differs from the paper’s grid baseline, so this is not reproduction of its numerical tables. Local timings and selected examples cannot establish superiority.

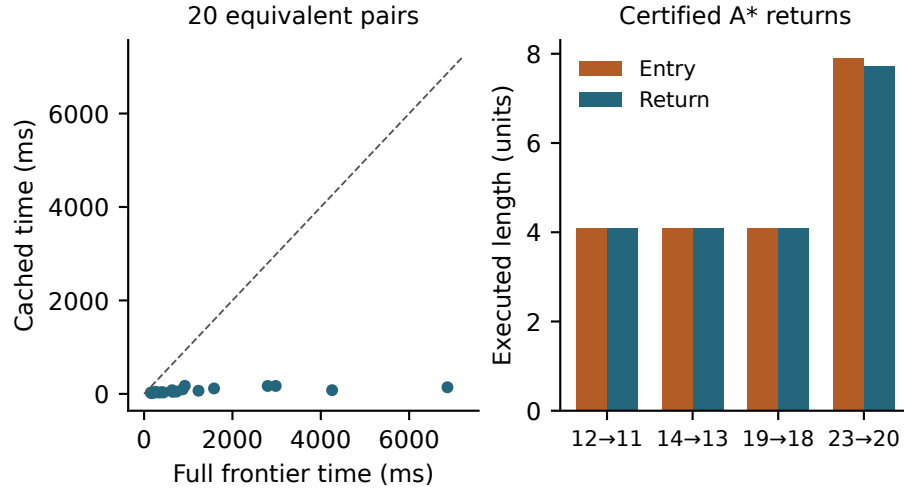


Fig. 8. Supporting evidence from our online simulator. Left: paired frontier processing with equivalent playback in 20 cases. Right: four observed indoor branch returns, each certified and marked nonfallback in the frozen trace. These are separate from the local authors-versus-A* comparison.

Sampling and timing. One road network, convenience robot maps, correlated decisions, overlapping held-out geometry, and 21 eligible requests limit external validity. Prospective selection reduces outcome bias without creating independent random trials. Scheduling, Python versions, compiler settings, batch averaging, and unmatched computation boundaries affect timing. Repetitions and ranges improve auditability without eliminating confounders; paired descriptive results support neither significance claims nor transferable speedup guarantees.

Reproducibility. Frozen records report four C++ suites, 169 Python tests, 34 frontend tests, and build/lint success, without proving optimal exploration. Report validation checks hashes and statistics without changing results.

9 Conclusion and Future Work

The information model determines the scope of path optimality. On known directed roads, the shared C++17 A* engine solves a fixed shortest-path problem. Under limited sensing, it solves successive observed-graph problems within an exploration controller. Reversible recorded entry and locked execution conditionally bound returns to observed ancestors, without globally optimizing unknown-world travel.

Road cost agreement accompanies substantial search-effort reductions on the tested network. Shared-state robot paths are primary-valid but have mixed lengths and boundary behavior, precluding general superiority. Timings identify

geometric construction costs. Online studies demonstrate cache savings with unchanged tested playback and genuine A*-computed certified returns, while retaining exploration regressions and nonmonotonic sensing outcomes.

The resulting implementation and evaluation provide a reproducible case study of graph search under different assumptions about environmental knowledge. Further work should examine independent maps, footprint-aware geometry, sensing error, and matched pipelines. Incremental replanning or richer geometric vertices require new controlled experiments.

References

1. Daniel, K., Nash, A., Koenig, S., Felner, A.: Theta*: Any-angle path planning on grids. *Journal of Artificial Intelligence Research* **39**, 533–579 (2010). <https://doi.org/10.1613/jair.2994>
2. Dijkstra, E.W.: A note on two problems in connexion with graphs. *Numerische Mathematik* **1**, 269–271 (1959). <https://doi.org/10.1007/BF01386390>
3. Elfes, A.: Using occupancy grids for mobile robot perception and navigation. *Computer* **22**(6), 46–57 (1989). <https://doi.org/10.1109/2.30720>
4. Hart, P.E., Nilsson, N.J., Raphael, B.: A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics* **4**(2), 100–107 (1968). <https://doi.org/10.1109/TSSC.1968.300136>
5. Kamon, I., Rivlin, E., Rimon, E.: A new range-sensor based globally convergent navigation algorithm for mobile robots. In: *Proceedings of the IEEE International Conference on Robotics and Automation*. vol. 1, pp. 429–435. IEEE (1996). <https://doi.org/10.1109/ROBOT.1996.503814>
6. Koenig, S., Likhachev, M.: D* Lite. In: *Proceedings of the Eighteenth National Conference on Artificial Intelligence*. pp. 476–483. AAAI Press (2002), <https://aaai-25.aaai.org/Papers/AAAI/2002/AAAI02-072.pdf>
7. Lozano-Pérez, T., Wesley, M.A.: An algorithm for planning collision-free paths among polyhedral obstacles. *Communications of the ACM* **22**(10), 560–570 (1979). <https://doi.org/10.1145/359156.359164>
8. Phan, T.A., Pham, H.A., Tran, T.B., Tran, V.H.: The sequences of bundles of line segments for autonomous robots with limited vision range to escape from blind alley regions. *Robotics and Autonomous Systems* **195**, 105185 (2026). <https://doi.org/10.1016/j.robot.2025.105185>
9. Yamauchi, B.: A frontier-based approach for autonomous exploration. In: *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation*. pp. 146–151. IEEE (1997). <https://doi.org/10.1109/CIRA.1997.613851>